

Pointer Arrays and Pointers to Pointers

Pointer Arrays and Pointers to Pointers

- ▶ Pointers can be stored in arrays just like other variables.
- ▶ Illustration: Sorting text lines into alphabetical order (a simplified version of UNIX sort).

Sorting Text Lines - Overview

- ▶ Three steps in the sorting process:
 1. Read all the lines of input.
 2. Sort them.
 3. Print them in order.
- ▶ Divide the program into functions for better structure.

Input and Output Routines

```
#include <stdio.h>
#include <string.h>
#define MAXLINES 5000
char *lineptr[MAXLINES];

int readlines(char *lineptr[], int nlines);
void writelines(char *lineptr[], int nlines);
```

The main function

```
int main() {
    int nlines;
    if ((nlines = readlines(lineptr, MAXLINES)) >= 0) {
        qsort(lineptr, 0, nlines-1);
        writelines(lineptr, nlines);
        return 0;
    } else {
        printf("error: input too big to sort\n");
        return 1;
    }
}
```

Input Routine - Readlines Function

```
#define MAXLEN 1000
int getline(char *, int);    char *alloc(int);
int readlines(char *lineptr[], int maxlines) {
    int len, nlines;
    char *p, line[MAXLEN];
    nlines = 0;
    while ((len = getline(line, MAXLEN)) > 0)
        if (nlines >= maxlines || (p = alloc(len)) == NULL)
            return -1;
        else {
            strcpy(p, line);
            lineptr[nlines++] = p;
        }
    return nlines;
}
```

Output Routine - Writelines Function

```
void writelines(char *lineptr[], int nlines) {  
    int i;  
    for (i = 0; i < nlines; i++)  
        printf("%s\n", lineptr[i]);  
}
```

Quicksort Function - Qsort

```
void qsort(char *v[], int left, int right) {  
    int i, last;  
    void swap(char *v[], int i, int j);  
    if (left >= right)  
        return;  
    swap(v, left, (left + right)/2);  
    last = left;  
    for (i = left+1; i <= right; i++)  
        if (strcmp(v[i], v[left]) < 0)  
            swap(v, ++last, i);  
    swap(v, left, last);  
    qsort(v, left, last-1);  
    qsort(v, last+1, right);  
}
```


Swap Function

```
void swap(char *v[], int i, int j) {  
    char *temp;  
    temp = v[i];  
    v[i] = v[j];  
    v[j] = temp;  
}
```

Multi-dimensional Arrays in C

- ▶ C provides rectangular multi-dimensional arrays.
- ▶ Often less used than arrays of pointers.
- ▶ Illustration through date conversion problem: day of the month to day of the year and vice versa.
- ▶ Eg., March 1 is the 60th day of a non-leap year, and the 61st day of a leap year.
- ▶ Two functions to do the conversions: `day_of_year` and `month_day`.
- ▶ `month_day(2024, 60, &m, &d)` sets `m` to 2 and `d` to 29 (February 29th).

Date Conversion Problem

```
static char daytab[2][13] = {  
    {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31},  
    {0, 31, 29, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}  
};
```

Day of the Year Function - day_of_year

```
int day_of_year(int year, int month, int day) {  
    int i, leap;  
    leap = year % 4 == 0 && year % 100 != 0 || year % 400 == 0;  
    for (i = 1; i < month; i++)  
        day += daytab[leap][i];  
    return day;  
}
```

Month and Day Function - month_day

```
void month_day(int year, int yearday, int *pmonth, int *pday) {  
    int i, leap;  
    leap = year % 4 == 0 && year % 100 != 0 || year % 400 == 0;  
    for (i = 1; yearday > daytab[leap][i]; i++)  
        yearday -= daytab[leap][i];  
    *pmonth = i;  
    *pday = yearday;  
}
```

Array Initialization and Access

- ▶ The array `daytab` is a two-dimensional array.
- ▶ Notational distinction between `daytab[i][j]` (correct in C) and `daytab[i,j]` (wrong in C).
- ▶ Elements are stored by rows, accessing them in storage order.
- ▶ Row major order

Function Parameter Passing

- ▶ If a two-dimensional array is passed to a function, the parameter declaration must include the number of columns.

```
void f(int daytab[2][13]) { ... }
```

or

```
void f(int daytab[][13]) { ... }
```

or

```
void f(int (*daytab)[13]) { ... }
```

Initialization of Pointer Arrays

- ▶ Function `month_name(n)`.
- ▶ Returns a pointer to a character string containing the name of the n -th month.
- ▶ Uses an internal static array for month names' initialization.

Month Name Function - month_name

```
char *month_name(int n) {  
    static char *name[] = {  
        "Illegal month",  
        "January", "February", "March",  
        "April", "May", "June",  
        "July", "August", "September",  
        "October", "November", "December"  
    };  
    return (n < 1 || n > 12) ? name[0] : name[n];  
}
```

Explanation

- ▶ `name` is an array of character pointers.
- ▶ The initializer is a list of character strings assigned to corresponding positions.
- ▶ Compiler determines the array size based on the number of initializers.

Pointers vs. Multi-dimensional Arrays

- ▶ Confusion between two-dimensional arrays and arrays of pointers.
- ▶ Example: `int a[10][20];` vs `int *b[10];`
- ▶ `a` is a 2-D array of 200 integers; `b` is an array of 10 integer pointers; each element of `b` must be initialized explicitly.
- ▶ Syntax for `a[3][4]` and `b[3][4]` is legal but behaves differently.
- ▶ `a[3][4]` is the 3rd row 4th column element of `a[3][4]`
- ▶ `b[3][4]` is the integer `4*sizeof(int)` bytes beyond where `b[3]` points; the same as `*(b[3]+4)`.

Pointers vs. Multi-dimensional Arrays (Contd.)

- ▶ Advantage of array of pointers: Rows may be of different lengths.
- ▶ Frequently used for storing character strings of diverse lengths.
- ▶ Example: `char *name[] = {"Illegal month", "Jan", "Feb", "Mar"};`
- ▶ Compare with a two-dimensional array: `char aname[][15] = {"Illegal month", "Jan", "Feb", "Mar"};`

Structures and Unions

Arrays of Structures

- ▶ Consider writing a program to count the occurrences of each C keyword.
- ▶ Arrays of parallel character strings and integers can be organized using an array of structures.
- ▶ Each keyword is a pair: `char *word; int count;`
- ▶ The structure declaration:

```
struct key {  
    char *word;  
    int count;  
} keytab[NKEYS];
```

Counting C Keywords Program

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
#define MAXWORD 100

struct key {
    char *word;
    int count;
};
```

Initialization of keytab

```
struct key keytab[] = {  
    {"auto", 0},  
    {"break", 0},  
    {"case", 0},  
    {"char", 0},  
    {"const", 0},  
    {"continue", 0},  
    {"default", 0},  
    /* ... */  
    {"unsigned", 0},  
    {"void", 0},  
    {"volatile", 0},  
    {"while", 0}  
};
```

- Initializers listed in pairs corresponding to structure members.

Number of Keywords: NKEYS

```
#define NKEYS (sizeof keytab / sizeof(struct key))
```

Uses the `sizeof` operator to compute the number of keywords. Ensures that the size of the array is completely determined at compile time. Another way:

```
#define NKEYS (sizeof keytab / sizeof(keytab[0]))
```

Advantage: This does not need to be changed if the type of `keytab` changes.

Counting C Keywords Program

```
int getword(char *, int);
int binsearch(char *, struct key *, int);

/* count C keywords */
main()
{
    int n;
    char word[MAXWORD];
    while (getword(word, MAXWORD) != EOF)
        if (isalpha(word[0]))
            if ((n = binsearch(word, keytab, NKEYS)) >= 0)
                keytab[n].count++;
    for (n = 0; n < NKEYS; n++)
        if (keytab[n].count > 0)
            printf("%4d %s\n", keytab[n].count, keytab[n].word);
    return 0;
}
```

Binary Search Function: binsearch

```
/* binsearch: find word in tab[0]...tab[n-1] */
int binsearch(char *word, struct key tab[], int n)
{
    int cond;
    int low, high, mid;
    low = 0;
    high = n - 1;
    while (low <= high) {
        mid = (low+high) / 2;
        if ((cond = strcmp(word, tab[mid].word)) < 0)
            high = mid - 1;
        else if (cond > 0)
            low = mid + 1;
        else
            return mid;
    }
    return -1;
}
```

Function: getword

```
int getword(char *word, int lim)
{
    /* getword: get next word or character from input */
    int c, getch(void);
    void ungetch(int);
    char *w = word;
    while (isspace(c = getch()))
        ;
    if (c != EOF)
        *w++ = c;
    if (!isalpha(c)) {
        *w = '\0';
        return c;
    }
}
```

Function: getword

```
for ( ; --lim > 0; w++)
    if (!isalnum(*w = getch())) {
        ungetch(*w);
        break;
    }
*w = '\0';
return word[0];
}
```

getword fetches the next “word”: either a string of letters and digits beginning with a letter, or a single non-white space character. Returns the first character of the word, or EOF for end of file, or the character itself if it is not alphabetic.

When the token stops, getword has gone one char too far. The call to ungetch pushes it back. getword also uses isspace to skip whitespace, isalpha to identify letters, and isalnum to identify letters and digits

Pointers to Structures

- ▶ Use pointers instead of array indices for enhanced flexibility.

Pointers to Structures

```
int getword(char *, int);
struct key *binsearch(char *, struct key *, int);

main() {
    char word[MAXWORD]; struct key *p;

    while (getword(word, MAXWORD) != EOF)
        if (isalpha(word[0]))
            if ((p=binsearch(word, keytab, NKEYS)) != NULL)
                p->count++;

    for (p = keytab; p < keytab + NKEYS; p++)
        if (p->count > 0)
            printf("%4d %s\n", p->count, p->word);

    return 0;
}
```

Pointers to Structures

```
struct key *binsearch(char *word, struct key *tab, int n) {
    int cond;
    struct key *low = &tab[0];
    struct key *high = &tab[n-1];
    struct key *mid;

    while (low < high) {
        mid = low + (high-low) / 2;

        if ((cond = strcmp(word, mid->word)) < 0)
            high = mid;
        else if (cond > 0)
            low = mid + 1;
        else
            return mid;
    }

    return NULL;
}
```